

Advanced RAG Concepts

Document ID	DOC008
Project	RAG
Category	RAG
Batch	batch_3

Purpose: Original educational notes created as a searchable PDF corpus for a portfolio-scale incremental RAG pipeline. The material is designed to contain meaningful technical sections that naturally produce multiple retrieval chunks.

Incremental RAG Ingestion

A RAG knowledge base does not need to be rebuilt whenever new documents arrive. An incremental ingestion pipeline identifies documents or chunks that have not been processed, generates embeddings only for that new content, and adds them to the existing vector store. This reduces API calls, processing time, and unnecessary duplication.

Practical note. In practice, the implementation should keep ingestion separate from querying, preserve source metadata, and inspect retrieved passages during testing rather than judging the final answer alone.

Idempotency with Document IDs and Hashes

A stable document identifier can indicate whether a source has already been processed. A content hash provides stronger detection because the same file name can contain changed content. A simple registry can store document ID, content hash, processing timestamp, and status. If an incoming document has an already successful ID and unchanged hash, the pipeline can skip it.

Practical note. In practice, the implementation should keep ingestion separate from querying, preserve source metadata, and inspect retrieved passages during testing rather than judging the final answer alone.

Persistent Vector Stores

A persistent vector database keeps embeddings and metadata between program runs. New vectors can be added without deleting existing ones. Chunk identifiers should be deterministic or otherwise unique so reruns do not accidentally create duplicate entries. Persistence separates ingestion from querying: users can query the existing collection without rerunning PDF extraction or embedding generation.

Practical note. In practice, the implementation should keep ingestion separate from querying, preserve source metadata, and inspect retrieved passages during testing rather than judging the final answer alone.

Retrieval Quality

Retrieval quality depends on document quality, chunking, embeddings, metadata, query wording, and the number of results returned. Poor retrieval cannot reliably be repaired by the generation model

because the model cannot ground an answer in information it never received. Testing should therefore inspect retrieved chunks separately from the final generated answer.

Practical note. In practice, the implementation should keep ingestion separate from querying, preserve source metadata, and inspect retrieved passages during testing rather than judging the final answer alone.

Handling Missing Context

A grounded RAG prompt should tell the language model what to do when the retrieved context does not contain enough information. A safe behavior is to state that the available documents do not provide the answer rather than filling gaps with unsupported details. This is especially important when RAG is used for internal policies or technical procedures.

Practical note. In practice, the implementation should keep ingestion separate from querying, preserve source metadata, and inspect retrieved passages during testing rather than judging the final answer alone.

Practical RAG Evaluation

A small portfolio evaluation can use predefined questions with expected topics or source documents. Checks can verify whether relevant chunks appear in top-k results, whether metadata filters constrain retrieval correctly, whether sources are returned, and whether the generated answer is supported by context. Complex evaluation frameworks are not required to demonstrate these fundamentals.

Practical note. In practice, the implementation should keep ingestion separate from querying, preserve source metadata, and inspect retrieved passages during testing rather than judging the final answer alone.

End of document.